**Name :** M.Sreekar

**Reg no :** 127156098

**Roll no :** 31 ( sec-F )

**Course :** Data Analytics in Health Care (DAHC)

# Ex - 5

## Extract Patient's Insights using NLP model from a medical text

## Aim :

To extract meaningful patient insights from medical text data using Natural Language Processing (NLP) techniques including text preprocessing, Named Entity Recognition (NER), sentiment analysis, and visualization.

## Objective :

The objective of this lab experiment is to employ natural language processing (NLP) techniques.to extract meaningful insights from medical text data. By analyzing clinical notes, patient records,and other textual healthcare data, the aim is to identify relevant information such as patient symptoms, diagnoses, treatments, and outcomes.

## Procedure :

### Medical Text Analysis Workflow

1. **Set Up**: Initialize Jupyter notebook and create a new notebook.
2. **Import Libraries**: Import necessary libraries:
   - `pandas`, `numpy` (Data handling)
   - `re`, `nltk`, `spacy` (NLP & Cleaning)
   - `transformers` (Hugging Face models)
   - `wordcloud`, `matplotlib` (Visualization)
3. **Load Data**: Load medical text data.
4. **Preprocess Text Data**: Clean and tokenize the text data.
5. **Named Entity Recognition (NER)**: Use **spaCy** to perform NER on the medical text data.
6. **Sentiment Analysis**: Use the **transformers** library to perform sentiment analysis.
7. **Generate Word Cloud**: Create a word cloud to visualize the most common terms.
8. **Extract Insights**: Combine extracted entities and sentiment results to generate patient insights.

# Step 0 : Import the necessary Libraries

```python
import pandas as pd
import numpy as np
import re
import nltk
import spacy
from transformers import pipeline
from nltk.corpus import  stopwords
from wordcloud import WordCloud
import matplotlib.pyplot as plt

# Download NLTK stopwords
nltk.download('stopwords')


# Load spaCy English model
nlp = spacy.load('en_core_web_sm')
```

[9] Python

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\admin\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

## 🛠️ NLP Pipeline: Tools and Libraries

### Library Requirements

- **pandas**: For data manipulation, cleaning, and creating DataFrames to store medical text.
- **numpy**: For numerical operations and handling array-based data (supporting library).
- **re (regex)**: For pattern-based text cleaning (removing special characters, numbers, and formatting).
- **nltk**: Provides the **stopwords** list to remove common words (like "the", "is", "at") that don't add meaning.
- **spaCy**: An advanced NLP library used here specifically for **Named Entity Recognition (NER)** to identify medical terms.
- **transformers**: The Hugging Face library used to load pre-trained models for **Sentiment Analysis**.
- **wordcloud**: Creates visual representations where the size of each word indicates its frequency.
- **matplotlib**: The foundation for displaying all visualizations and plots.

### Inference

These libraries form a complete **end-to-end NLP pipeline**, moving from raw text preprocessing and cleaning to advanced entity extraction, emotional analysis, and final visual insights.

# Step 1 : Load the Dataset (Medical Text)

Create Sample Medical Text Dataset

Generate + Code + Markdown

```python
# Sample Medical text dat representing patient records

medical_texts = [
    "Patient has a history of hypertension and diabetes. Prescrib
    "Asthma diagnosis confirmed. Patient advised to use inhaler d
    "Hypertension patient. Needs regular monitoring of blood pres
    "Diabetes patient. Recommended diet and exercise.",
    "Patient diagnosed with hypertension. Medication Y prescribed
]

#Create it into a DataFrame
medical_df = pd.DataFrame({'MedicalText': medical_texts})
print("Original Medical Texts:")
(medical_df.head())
```

[10] Python

```
Original Medical Texts:
```

| | MedicalText |
|---|---|
| 0 | Patient has a history of hypertension and diab... |
| 1 | Asthma diagnosis confirmed. Patient advised to... |
| 2 | Hypertension patient. Needs regular monitoring... |
| 3 | Diabetes patient. Recommended diet and exercise. |

## 📊 Data Structuring & Tabulation

### Need:

- **Structured Format**: Creates a organized data structure required for systematic and automated processing.
- **Granularity**: Ensures each row represents a unique patient record, clinical note, or observation for individual analysis.
- **Efficiency**: The **DataFrame** format enables high-performance, column-wise operations (like cleaning or sentiment scoring) across the entire dataset simultaneously.

### Inference:

Converting unstructured medical text into a **tabular format** is a critical first step. It allows for efficient batch processing and provides a clear audit trail of every transformation applied to each specific medical record.

---

# Step 2 : Preprocessing the Text

```python
def preprocess_text(text) :
    """
    Clean and standardize medical text for analysis

    Steps:
    1. Convert to lowercase
    2. Remove numbers
    3. Remove special characters/punctuation
    4. Remove stopwords

    """
    #convert to lower case
    text = text.lower()

    #Remove number
    text = re.sub( r'\d+', '', text)

    #remove special(non-word) characters (punctuation, symbols)
    text = re.sub(r'/W+',' ', text)

    #Remove stopwords

    text = ' '.join([word for word in text.split()
                     if word not in stopwords.words('english')])

    return text
```

[11] Python

```python
medical_df.columns
```

[12] Python

```
Index(['MedicalText'], dtype='object')
```

```python
# Apply preprocessing to all the ,medical texts

medical_df['CleanedText'] = medical_df['MedicalText'].apply(prepr

print("\n Precprocessed Text : ")
medical_df[['MedicalText', 'CleanedText']].head()
```

[29] Python

```
Precprocessed Text :
```

| | MedicalText | CleanedText |
|---|---|---|
| 0 | Patient has a history of hypertension and diab... | patient history hypertension diabetes. prescri... |
| 1 | Asthma diagnosis confirmed. Patient advised to... | asthma diagnosis confirmed. patient advised us... |
| 2 | Hypertension patient. Needs regular monitoring... | hypertension patient. needs regular monitoring... |
| 3 | Diabetes patient. Recommended diet and exercise. | diabetes patient. recommended diet exercise. |
| 4 | Patient diagnosed with hypertension. Medicatio... | patient diagnosed hypertension. medication pre... |

## 🧹 Text Preprocessing & Cleaning

### Need:

- **Lowercasing**: Ensures consistency across the dataset (e.g., treating "Diabetes" and "diabetes" as the same token).
- **Remove Numbers**: Strips out numeric data that may be irrelevant for general linguistic or thematic analysis.
- **Remove Special Characters**: Eliminates punctuation and symbols that do not contribute to core meaning extraction.
- **Remove Stopwords**: Filters out high-frequency but low-meaning words (e.g., "the", "and", "is") to focus on significant medical terms.

### Inference:

Preprocessing **standardizes the text** and drastically reduces noise. By retaining only meaningful words, we ensure higher accuracy for downstream tasks like entity recognition and sentiment analysis.

# Step 3 : Named Entity Recognition (NER) for Medical Entity Extraction

```python
import spacy
from spacy.pipeline import EntityRuler
import pandas as pd

# Load spaCy model
nlp = spacy.load("en_core_web_sm")

# Create EntityRuler and add patterns (Method 1: Add before addin
patterns = [
    {"label": "CONDITION", "pattern": [{"LOWER": "hypertension"}]
    {"label": "CONDITION", "pattern": [{"LOWER": "diabetes"}]},
    {"label": "CONDITION", "pattern": [{"LOWER": "asthma"}]},
    {"label": "MEDICATION", "pattern": [{"LOWER": "medication"},
    {"label": "MEDICATION", "pattern": [{"LOWER": "medication"},
    {"label": "MEDICATION", "pattern": [{"LOWER": "inhaler"}]}
]

# Method 1: Add ruler with config
ruler = nlp.add_pipe("entity_ruler", before="ner")
ruler.add_patterns(patterns)
```

[14] Python

```python
def extract_entities(text):
    """
    Extract named entities from medical text using spaCy

    Returns list of (entity_text, entity_type) tuples
    """
    doc = nlp(text)
    entities = [(ent.text, ent.label_) for ent in doc.ents]
    return entities

# Apply NER to original medical texts
medical_df['Entities'] = medical_df['MedicalText'].apply(extract_

print("\nExtracted Entities:")
for idx, row in medical_df.iterrows():
    print(f"\nText {idx+1}: {row['MedicalText']}")
    print(f"Entities: {row['Entities']}")

medical_df.head()
```

[15] Python

```
Extracted Entities:

Text 1: Patient has a history of hypertension and diabetes. Prescribe
Entities: [('hypertension', 'CONDITION'), ('diabetes', 'CONDITION')]

Text 2: Asthma diagnosis confirmed. Patient advised to use inhaler da
Entities: [('Asthma', 'CONDITION'), ('inhaler', 'MEDICATION'), ('dail

Text 3: Hypertension patient. Needs regular monitoring of blood press
Entities: [('Hypertension', 'CONDITION')]

Text 4: Diabetes patient. Recommended diet and exercise.
Entities: [('Diabetes', 'CONDITION')]
```

```
Text 5: Patient diagnosed with hypertension. Medication Y prescribed.
Entities: [('hypertension', 'CONDITION'), ('Medication Y', 'MEDICATI(
```

| | MedicalText | CleanedText | Entities |
|---|---|---|---|
| 0 | Patient has a history of hypertension and diab... | patient history hypertension diabetes. prescri... | [(hypertension, CONDITION), (diabetes, CONDITI... |
| 1 | Asthma diagnosis confirmed. Patient advised to... | asthma diagnosis confirmed. patient advised us... | [(Asthma, CONDITION), (inhaler, MEDICATION), (... |
| 2 | Hypertension patient. Needs regular monitoring... | hypertension patient. needs regular monitoring... | [(Hypertension, CONDITION)] |
| 3 | Diabetes patient. Recommended diet and exercise. | diabetes patient. recommended diet exercise. | [(Diabetes, CONDITION)] |
| 4 | Patient diagnosed with hypertension. Medicatio... | patient diagnosed hypertension. medication pre... | [(hypertension, CONDITION), (Medication Y, MED... |

## 🔍 Named Entity Recognition (NER)

### Need:

- **Condition Identification**: Automatically detects medical conditions such as **hypertension**, **diabetes**, or **asthma**.
- **Medication Detection**: Isolates pharmaceutical mentions like **medication X**, **inhalers**, or **medication Y**.
- **Treatment Extraction**: Recognizes specific treatment recommendations and clinical protocols.
- **Structured Extraction**: Transforms unstructured clinical narratives into a structured data format for further analysis.

### Inference:

NER serves as an automated "medical highlighter," identifying and classifying key information. This significantly reduces **manual review time** and enables the automated extraction of actionable insights from large volumes of patient notes.

# Step 4 : Sentiment Analysis

[16]

```python
!pip install torch torchvision torchaudio
```

Python

```
Requirement already satisfied: torch in c:\users\admin\anaconda3\envs
Requirement already satisfied: torchvision in c:\users\admin\anaconda
Requirement already satisfied: torchaudio in c:\users\admin\anaconda3
Requirement already satisfied: filelock in c:\users\admin\anaconda3\e
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users\
Requirement already satisfied: sympy>=1.13.3 in c:\users\admin\anaco
Requirement already satisfied: networkx>=2.5.1 in c:\users\admin\ana
Requirement already satisfied: jinja2 in c:\users\admin\anaconda3\en
Requirement already satisfied: fsspec>=0.8.5 in c:\users\admin\anaco
Requirement already satisfied: numpy in c:\users\admin\anaconda3\envs
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in c:\users\adm
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\admin\
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\admin\ana
```

```python
from transformers import pipeline
# Load the pre-trained sentiemt analysis pipeline
sentiement_pipeline = pipeline('sentiment-analysis')

def analyze_sentiment(text) :
    """
    Perform sentiment analysis on medical text

    Returns: Dictionary with 'label' and 'score'
    """
    result = sentiement_pipeline(text)[0]
    return result

# Apply sentiment Analysis
medical_df['Sentiment'] = medical_df['MedicalText'].apply(analyze

print("\nSentiment Analysis Results:")
for idx, row in medical_df.iterrows():
    sentiment = row['Sentiment']
    print(f"\nText {idx+1}: {row['MedicalText']}")
    print(f"Sentiment: {sentiment['label']} (Confidence: {sentime
```

[17] Python

```
No model was supplied, defaulted to distilbert/distilbert-base-uncase
Using a pipeline without specifying a model name and revision in proc
```

```
Could not render content for 'application/vnd.jupyter.widget-view+json'
{"model_id":"5ddedee165ab4442b975d3cf10f1543a","version_major":2,"version_minc
```

```
Sentiment Analysis Results:

Text 1: Patient has a history of hypertension and diabetes. Prescribe
Sentiment: NEGATIVE (Confidence: 0.962)

Text 2: Asthma diagnosis confirmed. Patient advised to use inhaler da
Sentiment: NEGATIVE (Confidence: 0.965)

Text 3: Hypertension patient. Needs regular monitoring of blood pres
Sentiment: NEGATIVE (Confidence: 0.997)

Text 4: Diabetes patient. Recommended diet and exercise.
Sentiment: POSITIVE (Confidence: 0.971)

Text 5: Patient diagnosed with hypertension. Medication Y prescribed.
Sentiment: NEGATIVE (Confidence: 0.968)
```

## 🧠 Sentiment Analysis

### Need:

- **Tone Identification**: Detects the underlying emotional tone within medical records and clinical narratives.
- **Psychological Insight**: Helps clinicians understand the patient's psychological state and overall outlook on their health.
- **Urgency Detection**: Flags high levels of concern or urgency in clinical notes that may require immediate follow-up.
- **Context Enrichment**: Complements objective clinical data with subjective emotional context for a holistic patient view.

### Inference:

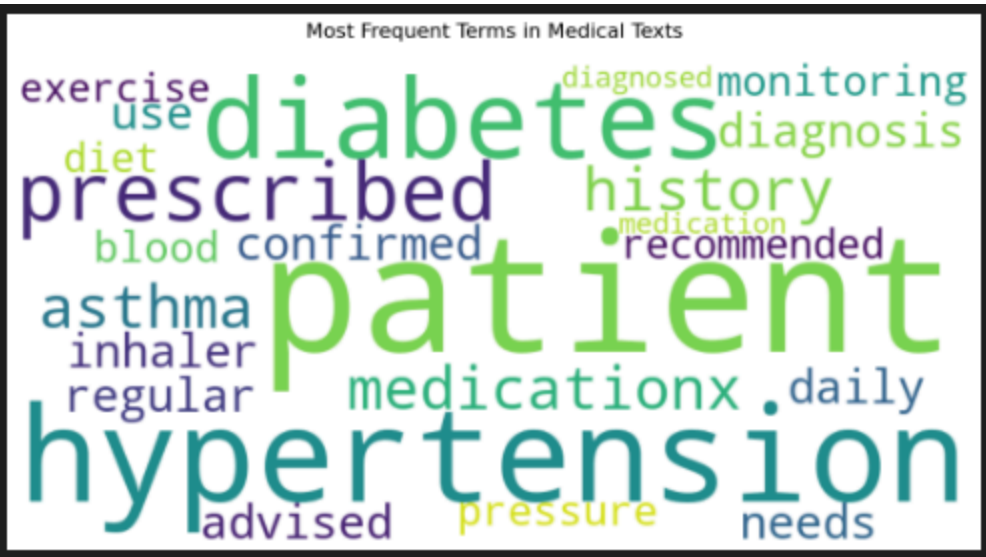
Even clinical texts carry emotional undertones. Sentiment analysis reveals critical indicators of **patient well-being** and **treatment satisfaction**, highlighting areas that may need additional support beyond physical care.

# Step 5 : Generate Word Cloud Visualization

```python
# Combine all the cleaned text for word cloud generation
all_text = ' '.join(medical_df['CleanedText'])

#Generate word cloud
wordcloud = WordCloud(
    width = 800,
    height = 400,
    background_color='white',
    colormap='viridis'
).generate(all_text)

# Display wordcloud
plt.figure(figsize=(12, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Most Frequent Terms in Medical Texts', fontsize=16, pa
plt.tight_layout()
plt.show()
```

Loading...

[18] Python




```python
# Display word frequency
from collections import Counter
words = all_text.split()
word_freq = Counter(words).most_common(10)
print("\nTop 10 Most Frequent Words:")
for word, count in word_freq:
    print(f"  {word}: {count}")
```

[19] Python

```
Top 10 Most Frequent Words:
  patient: 3
  hypertension: 2
  patient.: 2
  history: 1
  diabetes.: 1
  prescribed: 1
  medicationx.: 1
  asthma: 1
  diagnosis: 1
```

## ☁️ Word Cloud Visualization

### Need:

- **Term Importance**: Provides a visual representation where the size of the text indicates the frequency and importance of specific terms.
- **Theme Identification**: Allows for the quick identification of dominant medical themes across the entire dataset.
- **Highlight Focus**: Surfaces frequently mentioned conditions and treatments at a single glance.
- **Dataset Validation**: Serves as a sanity check to ensure the data aligns with the intended medical domain.

### Inference:

The word cloud serves as a visual confirmation of our focus; it should prominently feature terms like **hypertension**, **diabetes**, and **asthma**, validating that the preprocessing and data loading were successful.

---

# Step 6 : Extract Comprehensive Patient Insights

```python
# Extract Insights

def extract_insights(row) :
    entities = row['Entities']
    sentiment = row['Sentiment']

    return {
        'Entities' : entities,
        'Sentiment' : sentiment['label'],
        'SentimentScore' : sentiment['score']
    }

medical_df['Insights'] = medical_df.apply(extract_insights,axis=1
medical_df[['MedicalText', 'Insights']].head()
```

[21] Python

| | MedicalText | Insights |
|---|---|---|
| 0 | Patient has a history of hypertension and diab... | {'Entities': [('hypertension', 'CONDITION'), (... |
| 1 | Asthma diagnosis confirmed. Patient advised to... | {'Entities': [('Asthma', 'CONDITION'), ('inhal... |
| 2 | Hypertension patient. Needs regular monitoring... | {'Entities': [('Hypertension', 'CONDITION')], ... |
| 3 | Diabetes patient. Recommended diet and exercise. | {'Entities': [('Diabetes', 'CONDITION')], 'Sen... |
| 4 | Patient diagnosed with hypertension. Medicatio... | {'Entities': [('hypertension', 'CONDITION'), (... |

EXTRA 👍

```python
def extract_insights(row):
    """
    Combine entities and sentiment into comprehensive insights
    """
    entities = row['Entities']
    sentiment = row['Sentiment']

    # Categorize entities by type
    medical_conditions = []
    medications = []
    others = []

    for entity_text, entity_type in entities:
        if entity_type in ['DISEASE', 'CONDITION']:
            medical_conditions.append(entity_text)
        elif entity_type in ['MEDICATION', 'DRUG']:
            medications.append(entity_text)
        else:
            others.append((entity_text, entity_type))

    return {
        'medical_conditions': medical_conditions,
        'medications': medications,
        'other_entities': others,
        'sentiment_label': sentiment['label'],
        'sentiment_confidence': round(sentiment['score'], 3)
    }

# Apply insight extraction
medical_df['Insights'] = medical_df.apply(extract_insights, axis=

print("\n" + "="*80)
print("FINAL PATIENT INSIGHTS")
print("="*80)
```

```python
for idx, row in medical_df.iterrows():
    print(f"\n--- PATIENT RECORD {idx+1} ---")
    print(f"Original Text: {row['MedicalText']}")

    insights = row['Insights']
    print(f"\n📋 Medical Conditions: {insights['medical_conditio
    print(f"💊 Medications: {insights['medications']}")
    print(f"📌 Other Entities: {insights['other_entities']}")
    print(f"😊 Sentiment: {insights['sentiment_label']} "
          f"(Confidence: {insights['sentiment_confidence']})")
```

[27] Python

```
...
    ======================================================================
    FINAL PATIENT INSIGHTS
    ======================================================================

    --- PATIENT RECORD 1 ---
    Original Text: Patient has a history of hypertension and diabetes. Pr

    📋 Medical Conditions: ['hypertension', 'diabetes']
    💊 Medications: []
    📌 Other Entities: []
    😊 Sentiment: NEGATIVE (Confidence: 0.962)

    --- PATIENT RECORD 2 ---
    Original Text: Asthma diagnosis confirmed. Patient advised to use inh

    📋 Medical Conditions: ['Asthma']
    💊 Medications: ['inhaler']
```

```
📌 Other Entities: [('daily', 'DATE')]
😊 Sentiment: NEGATIVE (Confidence: 0.965)

--- PATIENT RECORD 3 ---
Original Text: Hypertension patient. Needs regular monitoring of bloc

📋 Medical Conditions: ['Hypertension']
...
📋 Medical Conditions: ['hypertension']
💊 Medications: ['Medication Y']
📌 Other Entities: []
😊 Sentiment: NEGATIVE (Confidence: 0.968)
```

*Output is truncated. View as a scrollable element or open in a text editor. Adjust cell out*

```python
# Aggregate insights across all patients
all_conditions = []
all_medications = []
sentiment_counts = {'POSITIVE': 0, 'NEGATIVE': 0}

for insights in medical_df['Insights']:
    all_conditions.extend(insights['medical_conditions'])
    all_medications.extend(insights['medications'])
    sentiment_counts[insights['sentiment_label']] += 1

print("\n" + "="*80)
print("AGGREGATED SUMMARY ACROSS ALL PATIENTS")
print("="*80)

print(f"\n📊 Total Patients Analyzed: {len(medical_df)}")
print(f"\n🩺 Medical Conditions Found:")
condition_freq = Counter(all_conditions).most_common()
for condition, count in condition_freq:
    print(f"   • {condition}: {count} patient(s)")

print(f"\n💊 Medications Prescribed:")
med_freq = Counter(all_medications).most_common()
for med, count in med_freq:
    print(f"   • {med}: {count} patient(s)")

print(f"\n😊 Sentiment Distribution:")
print(f"   • Positive: {sentiment_counts['POSITIVE']} patients")
print(f"   • Negative: {sentiment_counts['NEGATIVE']} patients")
```

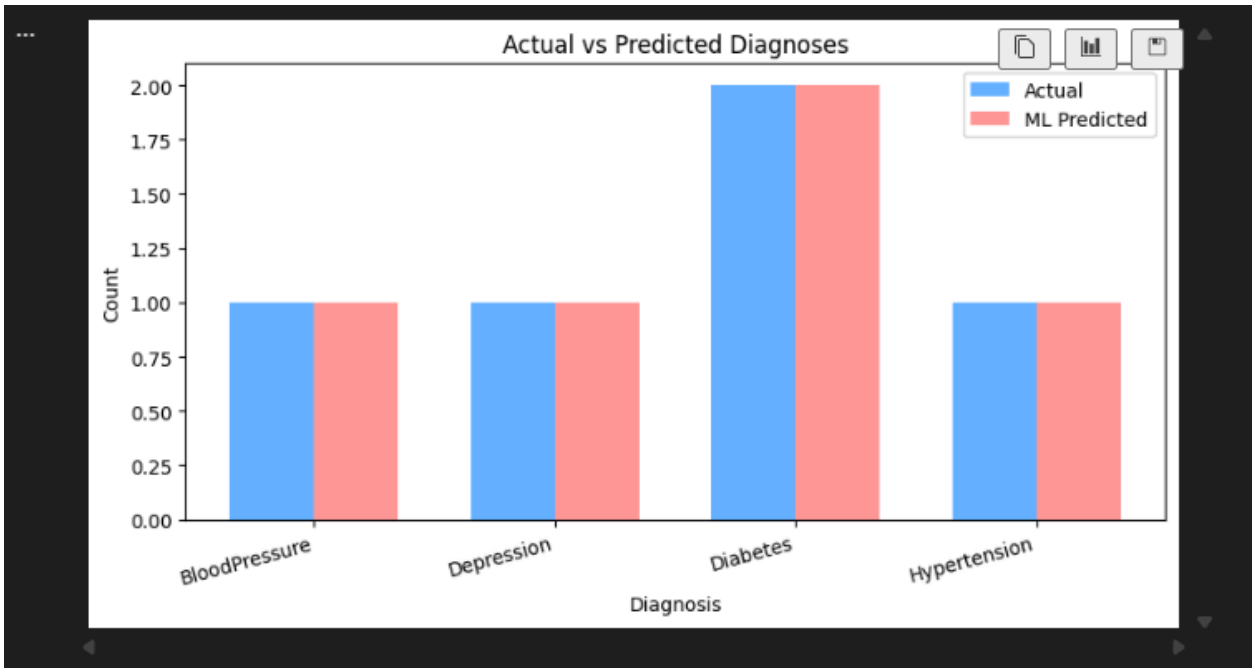
[28] Python

```
================================================================
AGGREGATED SUMMARY ACROSS ALL PATIENTS
================================================================

📊 Total Patients Analyzed: 5

🏥 Medical Conditions Found:
   • hypertension: 2 patient(s)
   • diabetes: 1 patient(s)
   • Asthma: 1 patient(s)
   • Hypertension: 1 patient(s)
   • Diabetes: 1 patient(s)

💊 Medications Prescribed:
   • inhaler: 1 patient(s)
   • Medication Y: 1 patient(s)

😊 Sentiment Distribution:
   • Positive: 1 patients
   • Negative: 4 patients
```

# Diagnosis Distribution – Actual vs ML Prediction

```python
from collections import Counter

actual_diags = [item[1] for item in ml_result]
pred_diags   = [item[2] for item in ml_result]

actual_count = Counter(actual_diags)
pred_count   = Counter(pred_diags)

all_diagnoses = sorted(set(actual_diags + pred_diags))

x = np.arange(len(all_diagnoses))
width = 0.35

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.bar(x - width/2, [actual_count.get(d, 0) for d in all_diagnoses], width, label='Actu
ax.bar(x + width/2, [pred_count.get(d, 0) for d in all_diagnoses], width, label='ML Pre

ax.set_xlabel('Diagnosis')
ax.set_ylabel('Count')
ax.set_title('Actual vs Predicted Diagnoses')
ax.set_xticks(x)
ax.set_xticklabels(all_diagnoses, rotation=15, ha='right')
ax.legend()

plt.tight_layout()
plt.show()
```

[14] ✓ 0.1s Python


Actual vs Predicted Diagnoses
Actual
ML Predicted
Count
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
BloodPressure
Depression
Diabetes
Hypertension
Diagnosis